Simulation Study for Pseudo-value Regression

FengQi (Cynthia) Song

2026-03-04

Contents

1 Objective **1**

1.1 Sanity check 1

1.2 Monte Carlo validation 2

1.3 Sensitivity analysis(different evaluation time) 3

1.4 Pseudo-value Regression Model 5

1.5 Type I error 5

1.6 Power 7

---

1 Objective

1.1 Sanity check

```
library(MCC)
source("01_generate_data.R")
t_eval <- 5
pseudo <- MCC::GenPseudo(data = dat, tau = t_eval)

str(pseudo)
```

```
## 'data.frame':    200 obs. of  3 variables:
##  $ idx   : num  1 2 3 4 5 6 7 8 9 10 ...
##  $ psi   : num  0.0744 -4.4227 -1.0431 -1.8381 2.4891 ...
##  $ pseudo: num  4.879 0.382 3.761 2.967 7.294 ...
```

```
head(pseudo)
```

```
##   idx         psi     pseudo
## 1   1  0.07442166  4.8790567
## 2   2 -4.42267421  0.3819608
## 3   3 -1.04314090  3.7614941
```

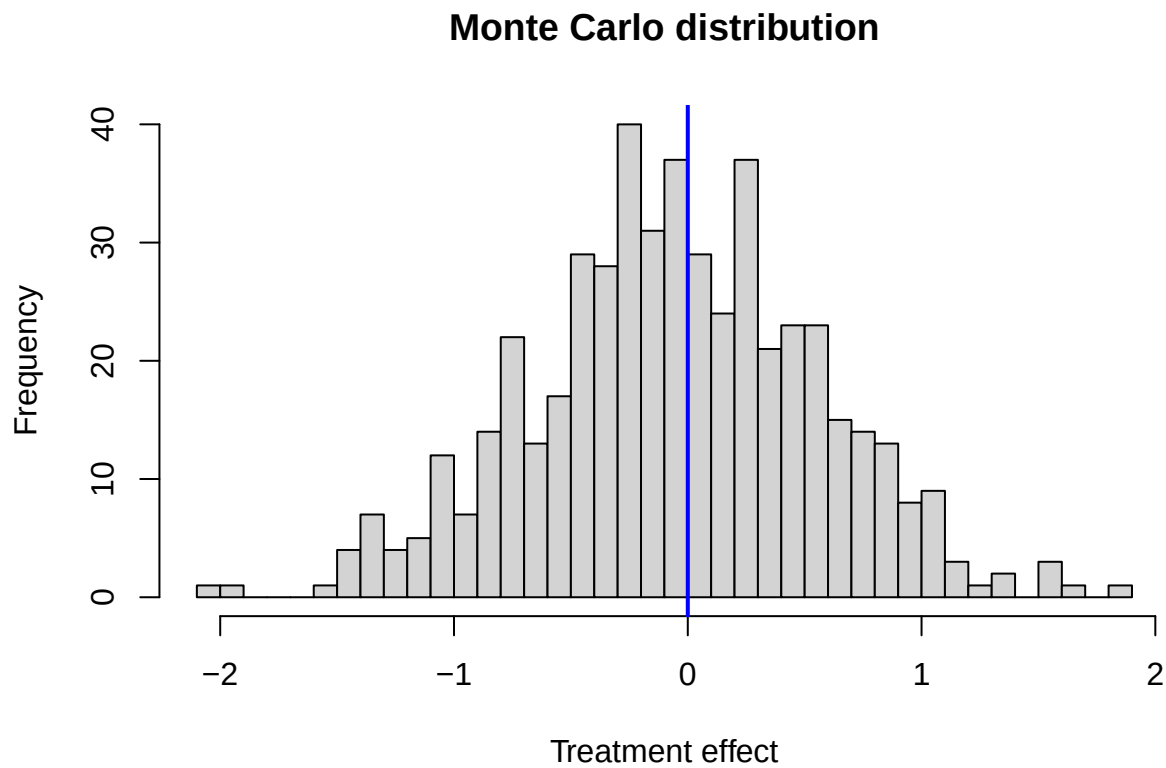
```
## 4   4 -1.83809885  2.9665362
## 5   5  2.48908415  7.2937192
## 6   6 -5.04139954 -0.2367645
```

```r
AX <- unique(dat[, c("idx", "A", "X")])
merged <- merge(AX, pseudo[, c("idx", "pseudo")], by = "idx")

tapply(merged$pseudo, merged$A, mean)
```

```
##        0        1
## 5.311600 4.297671
```

1.2 Monte Carlo validation

```r
library(MCC)
source("01_generate_data.R")

set.seed(123)

B <- 500
delta_hat <- numeric(B)

for (b in 1:B) {
  source("01_generate_data.R")
  pseudo <- MCC::GenPseudo(data = dat, tau = 5)

  AX <- unique(dat[, c("idx", "A")])
  merged <- merge(AX, pseudo[, c("idx", "pseudo")], by = "idx")

  delta_hat[b] <- mean(merged$pseudo[merged$A == 1]) -
                  mean(merged$pseudo[merged$A == 0])
}

hist(delta_hat, breaks = 30,
     main = "Monte Carlo distribution",
     xlab = "Treatment effect")
abline(v = 0, col = "blue", lwd = 2)
```



This distribution is centered at zero, suggesting no bias

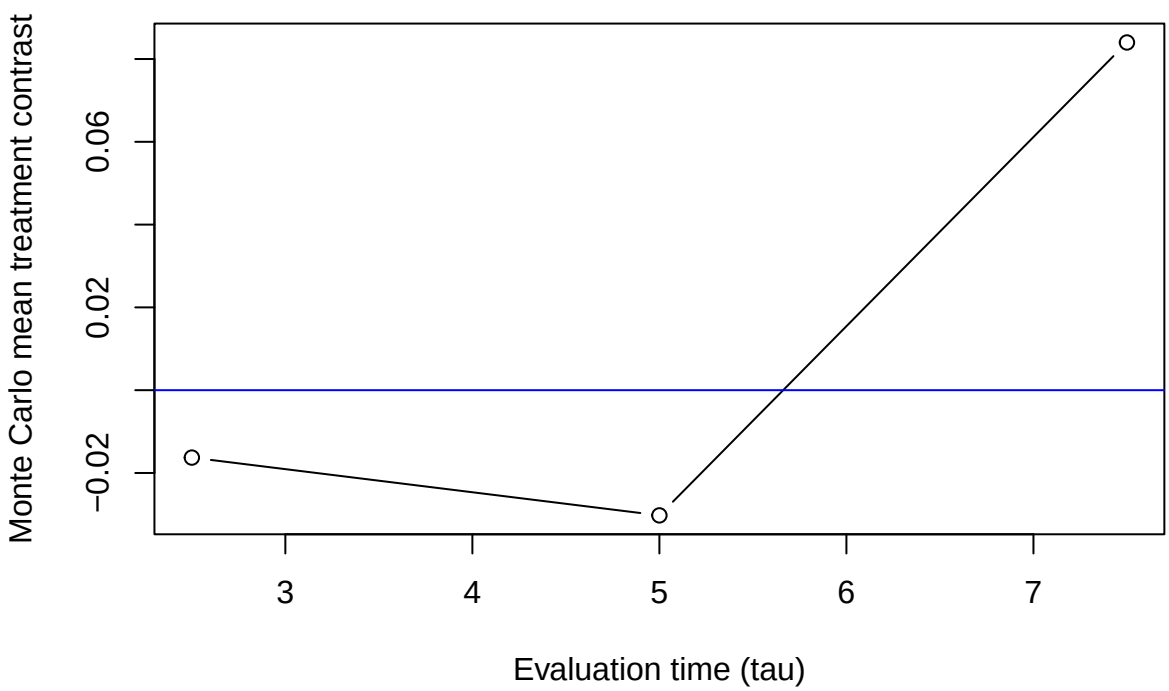
1.3 Sensitivity analysis(different evaluation time)

```
set.seed(123)

B <- 500
t_list <- c(2.5, 5, 7.5)

out_time <- data.frame(
  t_eval = t_list,
  mean_diff = NA_real_,
  sd_diff = NA_real_
)

for (j in seq_along(t_list)) {

  t_eval <- t_list[j]
  delta_tau <- numeric(B)

  for (b in 1:B) {   #Suggested by Generative AI

    source("01_generate_data.R")
```

```r
    pseudo_j <- MCC::GenPseudo(data = dat, tau = t_eval)
    AX <- unique(dat[, c("idx", "A")])
    merged_j <- merge(AX, pseudo_j[, c("idx", "pseudo")], by = "idx")

    delta_tau[b] <- mean(merged_j$pseudo[merged_j$A == 1]) -
                    mean(merged_j$pseudo[merged_j$A == 0])
  }

  out_time$mean_diff[j] <- mean(delta_tau)
  out_time$sd_diff[j]   <- sd(delta_tau)
}

out_time
```

```
##   t_eval   mean_diff   sd_diff
## 1    2.5 -0.01628350 0.2077605
## 2    5.0 -0.03023934 0.6158638
## 3    7.5  0.08399123 1.1209625
```

```r
plot(out_time$t_eval, out_time$mean_diff, type = "b",
     xlab = "Evaluation time (tau)",
     ylab = "Monte Carlo mean treatment contrast")
abline(h = 0, col = "blue")
```



1.4 Pseudo-value Regression Model

```
# Single-run regression across tau

t_list <- c(2.5, 5, 7.5)
AX <- unique(dat[, c("idx", "A")])

reg_out <- data.frame(
  t_eval = t_list,
  beta_A = NA,
  p_A = NA
)

for (j in seq_along(t_list)) {
  t_eval <- t_list[j]
  pseudo_j <- MCC::GenPseudo(data = dat, tau = t_eval)
  reg_dat <- merge(AX, pseudo_j[, c("idx", "pseudo")], by = "idx")

  fit <- lm(pseudo ~ A, data = reg_dat)
  s <- summary(fit)$coefficients

  reg_out$beta_A[j] <- s["A", "Estimate"]
  reg_out$p_A[j] <- s["A", "Pr(>|t|)"]
}

reg_out
```

```
##   t_eval     beta_A       p_A
## 1    2.5 -0.2106157 0.2445820
## 2    5.0  0.3965026 0.4724593
## 3    7.5  0.9282359 0.3961466
```

1.5 Type I error

```
set.seed(123)

B <- 500
alpha <- 0.05
t_list <- c(2.5, 5, 7.5)

type1_result <- data.frame(
  t_eval = t_list,
  reject_rate = NA,
  mean_beta = NA
)

for (j in seq_along(t_list)) {

  t_eval <- t_list[j]
  reject <- logical(B)
```

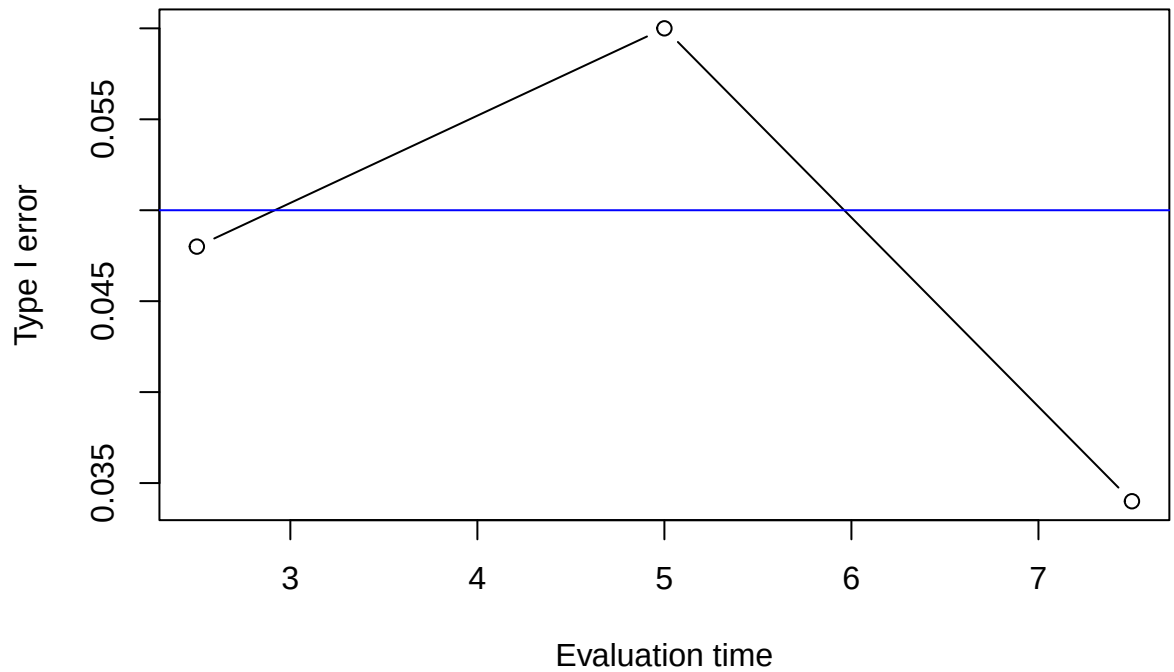
```r
  beta <- numeric(B)

  for (b in 1:B) {

    alphaA <- 0
    source("01_generate_data.R")

    pseudo_b <- MCC::GenPseudo(data = dat, tau = t_eval)

    AX <- unique(dat[, c("idx", "A")])
    reg_dat <- merge(AX, pseudo_b[, c("idx", "pseudo")], by = "idx")

    fit <- lm(pseudo ~ A, data = reg_dat)
    s <- summary(fit)$coefficients

    beta[b] <- s["A", "Estimate"]
    reject[b] <- s["A", "Pr(>|t|)"] < alpha
  }

  type1_result$reject_rate[j] <- mean(reject)
  type1_result$mean_beta[j] <- mean(beta)
}


type1_result
```

```
##   t_eval reject_rate   mean_beta
## 1    2.5       0.048 -0.01628350
## 2    5.0       0.060 -0.03023934
## 3    7.5       0.034  0.08399123
```

```r
type1_result
```

```
##   t_eval reject_rate   mean_beta
## 1    2.5       0.048 -0.01628350
## 2    5.0       0.060 -0.03023934
## 3    7.5       0.034  0.08399123
```

```r
plot(type1_result$t_eval, type1_result$reject_rate, type = "b",
     xlab = "Evaluation time",
     ylab = "Type I error")
abline(h = 0.05, col = "blue")
```



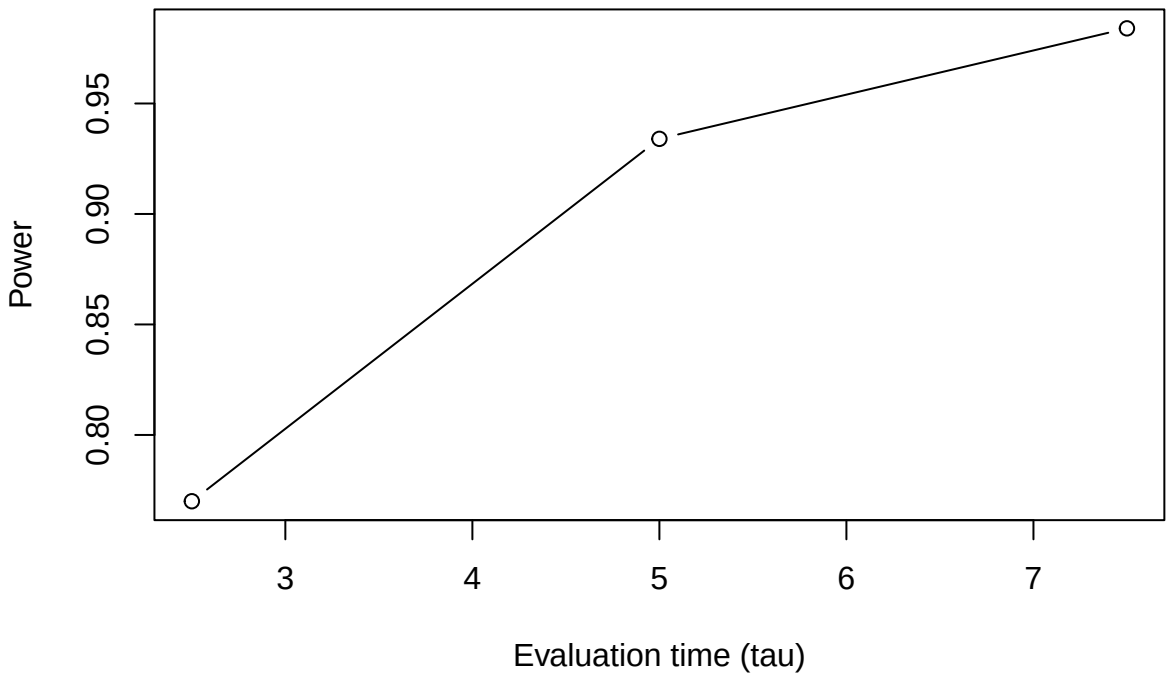
1.6 Power

```
# Type II error
set.seed(123)

B <- 500
alpha <- 0.05
t_list <- c(2.5, 5, 7.5)

alphaA <- log(1.5)

power_out <- data.frame(
  t_eval = t_list,
  power = NA_real_,
  mean_beta = NA_real_
)

for (j in seq_along(t_list)) {

  t_eval <- t_list[j]
  reject <- logical(B)
  beta <- numeric(B)

  for (b in 1:B) {
```

```r
    source("01_generate_data.R")



    pseudo_b <- MCC::GenPseudo(data = dat, tau = t_eval)
    AX <- unique(dat[, c("idx", "A")])
    reg_dat <- merge(AX, pseudo_b[, c("idx", "pseudo")], by = "idx")

    fit <- lm(pseudo ~ A, data = reg_dat)
    s <- summary(fit)$coefficients

    beta[b] <- s["A", "Estimate"]
    reject[b] <- s["A", "Pr(>|t|)"] < alpha
  }

  power_out$power[j] <- mean(reject)
  power_out$mean_beta[j] <- mean(beta)
}

power_out
```

```
##   t_eval power mean_beta
## 1    2.5 0.770 0.6316979
## 2    5.0 0.934 2.4367397
## 3    7.5 0.984 5.5175678
```

```r
plot(power_out$t_eval, power_out$power, type = "b",
     xlab = "Evaluation time (tau)",
     ylab = "Power")
abline(h = alpha, col = "blue")
```


Power
0.80
0.85
0.90
0.95
3
4
5
6
7
Evaluation time (tau)